Solution: Graph Valid Tree

Problem Statement

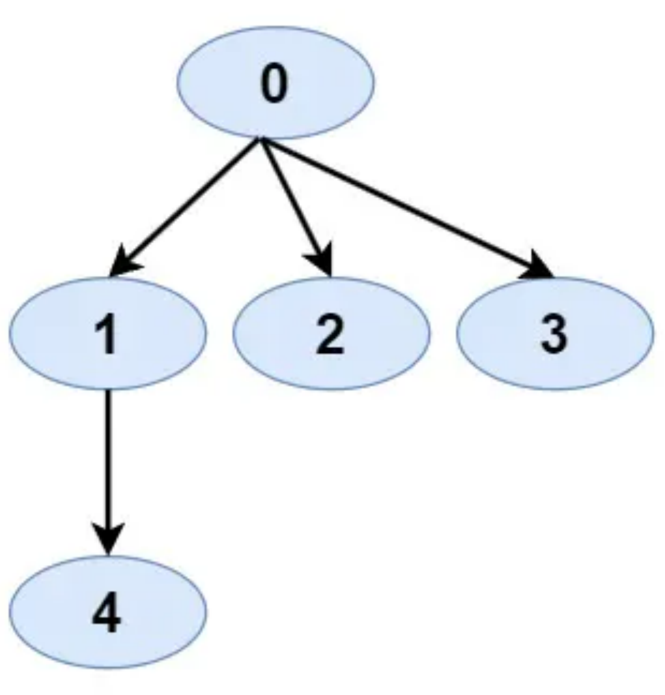
Given a number `n`, which indicates the number of nodes numbered from `0` to `n-1`, and a list of undirected edges for the graph, determine if the graph is a valid tree.

A graph qualifies as a valid tree if it meets the following criteria:

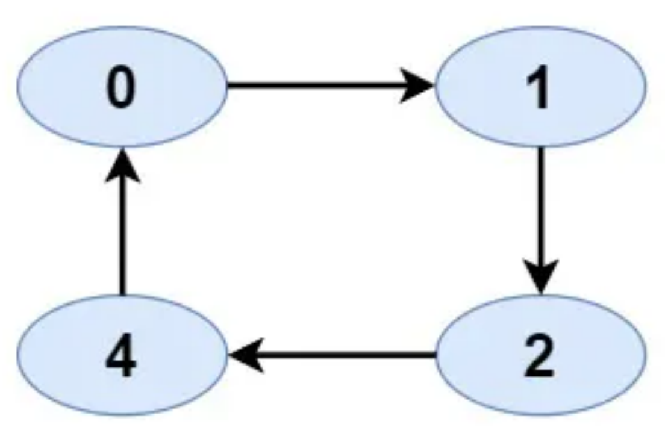
1. It has no cycles.
2. It is fully connected.

Examples

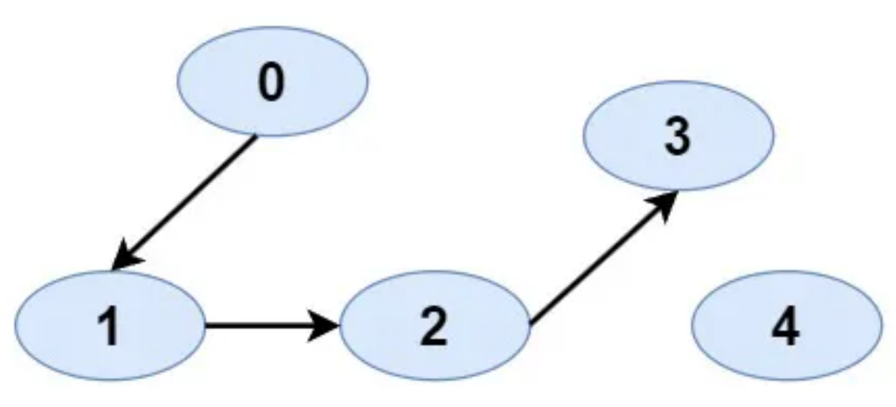
- **Example 1**:
  - **Input**: n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]



- **Expected Output**: true
- **Justification**: There are no cycles in the graph, and all nodes are connected, forming a valid tree.
- **Example 2**:
  - **Input**: n = 4, edges = [[0,1],[1,2],[2,3],[3,0]]]



- **Expected Output**: false
- **Justification**: There is a cycle in the graph (0-1-2-3-0), thus it's not a valid tree.
- **Example 3**:
  - **Input**: n = 5, edges = [[0,1],[1,2],[2,3]]



- **Expected Output**: false
- **Justification**: TNode 4 is not connected to any other node, making the graph not fully connected.

Solution

To determine if a given undirected graph forms a valid tree, we must ensure two key conditions are met:

1. The graph should not have any cycles.
2. The graph should be fully connected (i.e., all nodes are reachable from any other node).

To verify these conditions, we can adopt the following approach:

1. **Initialization**: Construct an adjacency list for the graph. Initialize a set (or an array) to keep track of visited nodes.
2. **DFS for Cycle Check**: Use Depth First Search (DFS) to explore the graph. During this traversal, if we ever try to revisit a node we've previously visited (excluding the parent node), it indicates the presence of a cycle.
3. **Connectivity Check**: After ensuring there are no cycles, we need to ensure that the graph is fully connected. This can be done by verifying that the number of unique nodes visited during our DFS equals the total number of nodes.
4. **Return Result**: If both checks pass, then it's a valid tree and we return true. Otherwise, return false.

Algorithm Walkthrough

Using the input from Example 1:

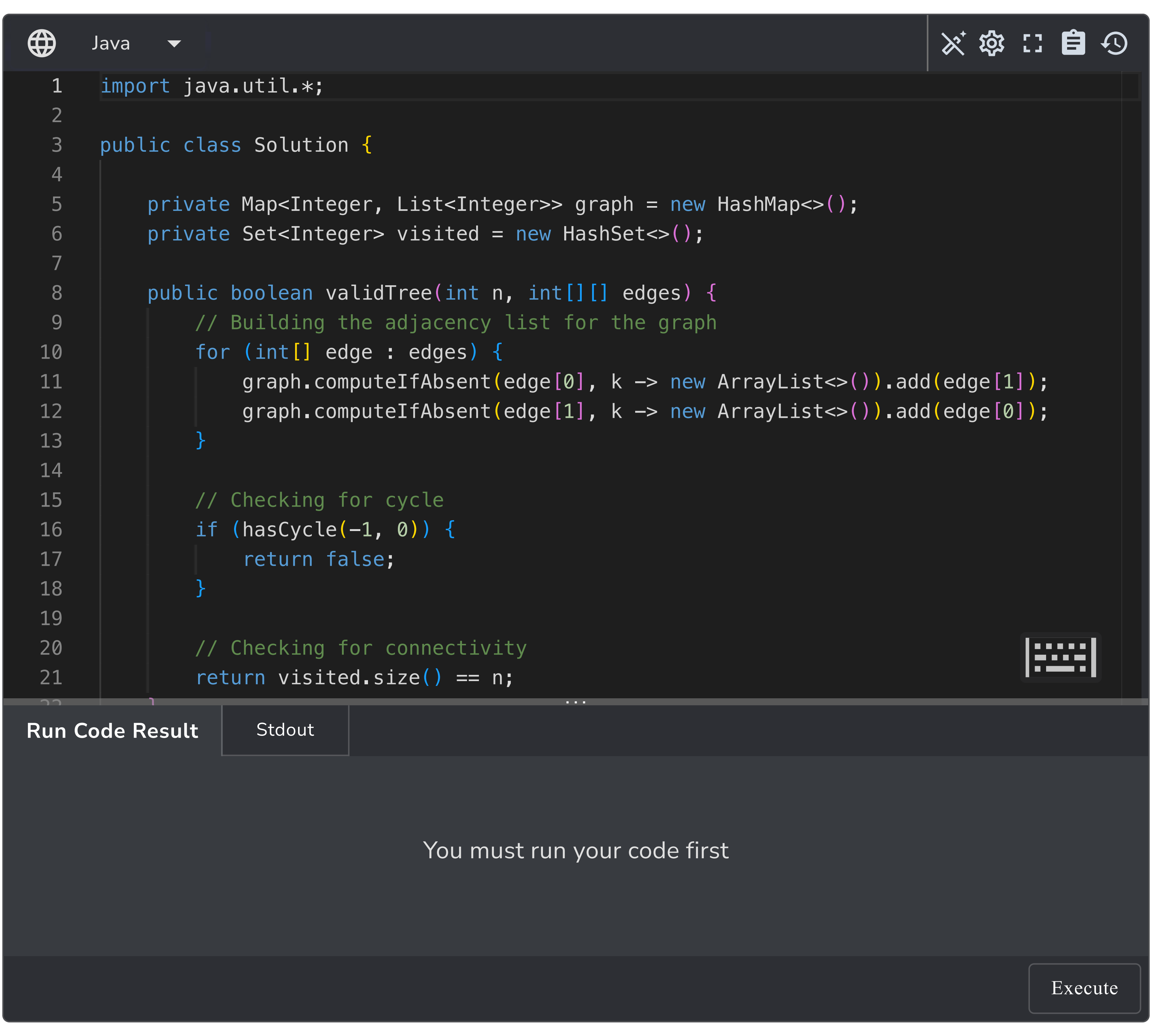
```
n = 5
edges = [[0,1],[0,2],[0,3],[1,4]]
```

- Start with the adjacency list:

  ```
  0 -> [1, 2, 3]
  1 -> [0, 4]
  2 -> [0]
  3 -> [0]
  4 -> [1]
  ```

- Start DFS from node 0. Mark node 0 as visited.
- Move to node 1 (from node 0). Mark node 1 as visited.
- From node 1, visit node 4. Mark node 4 as visited.
- Backtrack to node 1, and backtrack again to node 0.
- From node 0, visit node 2. Mark node 2 as visited.
- From node 0, visit node 3. Mark node 3 as visited.
- At the end of DFS, we've visited all nodes and found no cycles.
- Since we visited all nodes, the graph is fully connected.
- Hence, the graph is a valid tree.

Code



```
import java.util.*;

public class Solution {

    private Map<Integer, List<Integer>> graph = new HashMap<>();
    private Set<Integer> visited = new HashSet<>();

    public boolean validTree(int n, int[][] edges) {
        // Building the adjacency list for the graph
        for (int[] edge : edges) {
            graph.computeIfAbsent(edge[0], k -> new ArrayList<>()).add(edge[1]);
            graph.computeIfAbsent(edge[1], k -> new ArrayList<>()).add(edge[0]);
        }

        // Checking for cycle
        if (hasCycle(-1, 0)) {
            return false;
        }

        // Checking for connectivity
        return visited.size() == n;
```

You must run your code first

Complexity Analysis

- **Time Complexity:** The algorithm visits each node and edge once, so its time complexity is (O(n + e)), where (n) is the number of nodes and (e) is the number of edges.
- **Space Complexity:** We are using a graph and visited set, so the space complexity is (O(n + e)).

← Previous
Graph Valid Tree (Medium)

Next →
Implement Trie (Prefix Tree) (Medium)

☐ Mark as Completed